

M16

DataOps, CI/CD, Security & Observability

Beginner-first production engineering learner book

This book is your teacher

Option B | Charcoal + Teal + Soft Blue | Standalone DE 2026

Contents / Index

DE16-01	Production mindset	p6
DE16-02	Testing strategy for data engineering	p9
DE16-03	Automated quality gates	p12
DE16-04	Continuous Integration fundamentals	p15
DE16-05	GitHub Actions for data projects	p18
DE16-06	Dev, test and prod environments	p21
DE16-07	Deployment, releases and rollback	p24
DE16-08	Secrets and credential safety	p27
DE16-09	RBAC and least privilege	p30
DE16-10	PII, encryption and masking	p33
DE16-11	Metadata, lineage and governance	p36
DE16-12	Observability for data pipelines	p39
DE16-13	Incidents, runbooks and root cause	p42
DE16-14	Terraform and IaC foundations	p45
	Mastery + revision	p48-49

Module Map

Purpose

Turn working data systems into controlled production systems: tested, change-safe, least-privilege, observable, recoverable and evidence-backed.

Production controls	DE16-01 to 03	operate, test by risk, block unsafe data
Safe change	DE16-04 to 07	CI, workflows, environments, release/rollback
Security/governance	DE16-08 to 11	secrets, RBAC, PII, metadata/lineage
Operations/IaC	DE16-12 to 14	observability, incidents, Terraform desired state

G06 relationship

M16 does NOT open G06. Continue to M17 Architecture / System Design after M16. G06 opens only after the complete M15 + M16 + M17 block.

Tools / First-use Map

Python 3 + pytest + PyYAML	local tests/policies	Use existing learning environment; install pytest/pyyaml only if missing.
Git/GitHub	workflow reading + optional live CI	M06 prerequisite. A local YAML file is not a live GitHub Actions run.
GitHub Actions/OIDC	CI/deployment identity	Read current docs. Keep disabled OIDC example disabled until an authorized cloud target exists.
Terraform 1.16.x stable line	IaC lesson	Use official HashiCorp install only. Artifact QA does not fabricate terraform runtime.
Cloud/Fabric/security tools	concept/live optional	M15 prerequisite; actual tenant/permissions remain live evidence only.

2026 OIDC note

For newer GitHub repositories, default OIDC subject behavior may include immutable owner/repository identifiers. Match trust policy to the actual current token claims and official documentation; do not memorize or paste one universal sub string.

How to study M16

Teacher loop

WHY -> OUTCOME -> START STATE -> PICTURE -> NEW WORDS -> SOURCE -> FOLLOW WITH ME -> EXPECTED -> WHY IT WORKED -> MISTAKES -> RECOVER -> YOUR TURN -> VALIDATE -> TEACH BACK -> PROTECTED REVIEW + CHANGED RETRY -> EVIDENCE/NEXT.

Evidence discipline

For every lesson, CREATE attempt.md only after the guided run. Review stays closed until the attempt and validation are saved. If weak, preserve attempt.md, open the protected Review, repair the smallest missing concept, close Review, then CREATE retry.md with changed proof. Never edit a failed attempt into a pass.

Runtime honesty

Local Python/YAML/pytest evidence may be executed in this package. A real GitHub Actions run, GitHub environment protection/OIDC exchange, cloud role assignment, and Terraform CLI/cloud plan are separate runtime claims. Record PENDING when not actually observed.

DE16-01 - Production mindset

1. Why this matters

Think like the engineer who must operate and recover a pipeline tomorrow, not only the person who made it run once. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Think like the engineer who must operate and recover a pipeline tomorrow, not only the person who made it run once.

3. Prerequisite / start state

You have completed cloud/Fabric foundations in M15. You can run Python locally and understand pipelines, evidence and failures.

4. Picture it in your head

Picture a restaurant kitchen on a busy night. A prototype is one meal cooked successfully once. Production is the whole kitchen delivering safely every day when ingredients arrive late, a worker is absent, an appliance fails, or demand spikes. Data production needs the same operating discipline: repeatable change, validation, ownership, monitoring, recovery and proof.

5. New words before we use them

prototype = one-off working version; production = service expected to keep working; control = check that reduces a known risk; SLO/SLA = target/commitment for service behavior; rollback = return to a known-good release; runbook = written recovery procedure.

6. Useful video / source route

[Luke Barousse - How I Would Learn to Be a Data Engineer \(2026\) - 27:38 orchestration: 30:01 terminal/bash/git: 32:36 DataOps/CI-CD: 34:10 architecture](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-01 - Follow with me once

7. Follow with me once

Open 04_PRACTICE_FILES/PARCELPULSE_PROJECT. First trace README_FIRST.md and the project tree without running anything. Point to tests/, src/quality_gate.py, src/secret_scan.py, src/observability.py, release/rollback_plan.md and governance/. Say aloud which risk each file controls. Then run `python src/pipeline.py data/orders_good.csv --out evidence/DE16-01_good`. Open metrics.json and curated_orders.csv. Mark this as execution evidence, not merely a screenshot. Finally imagine orders_bad.csv arrived after deployment: decide which control should block publication, which signal should alert the owner, and which recovery artifact tells you what to do next. The point is to connect code to an operating system of controls, not to memorize folder names.

```
input -> validate -> transform -> quality gate -> publish
      |
      +--> metrics/alerts -> incident -> recover
```

8. What you should see / be able to say

A successful good-data run plus a written control chain: change -> test -> quality/security -> deploy -> observe -> incident/recovery. You can name one blocking failure, one operational signal and one recovery artifact.

9. Why it worked

The pipeline produced output because its code worked; the production argument is stronger because separate controls can detect bad input, exposed secrets, stale data and failed deployment. Production readiness is about predictable behavior under both success and failure.

10. Common beginner mistakes

Calling a laptop success production-ready; adding monitoring only after launch; treating recovery as “rerun it”; having no named owner; saving only happy-path screenshots.

11. If you get stuck

If the project feels like too many files, start with README_FIRST.md and draw only five boxes: input, pipeline, quality, monitoring, recovery. Add one file to each box. Run only the good pipeline. If Python fails, check your current folder and `python --version` before changing code.

DE16-01 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-01/attempt.md. Draw your own production-readiness chain for a parcel pipeline and classify at least six existing project artifacts as pre-deploy, deploy, post-deploy or recovery evidence. Do not copy the diagram above.

13. Validate

Your attempt names a risk, an observable signal, an owner/action and a preserved evidence file for each major stage. Re-run the good pipeline and record the actual exit/result path.

14. Teach it back

Explain in 60-90 seconds why “the code ran once” is weaker than “the service is production-ready,” using one concrete ParcelPulse failure.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: For ClinicOps, design the production control chain for a 20-minute freshness target and stricter privacy. Change the domain, failure and recovery evidence; do not reuse ParcelPulse wording.

16. Evidence / mastery + exact next route

attempt.md + actual local run output. Live cloud/CI evidence is not required for this lesson. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-02.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-02 - Testing strategy for data engineering

1. Why this matters

Design a layered testing strategy that maps tests to risks instead of maximizing test count. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Design a layered testing strategy that maps tests to risks instead of maximizing test count.

3. Prerequisite / start state

You can run pytest and read simple Python tests from M04/M05. ParcelPulse contains 9 executable tests.

4. Picture it in your head

A vehicle inspection does not use one giant test. A mechanic checks a part, then connected systems, then road behavior. Data systems need the same layers: tiny logic tests, data-contract tests, component/integration tests, end-to-end checks, and production observations. Each layer catches a different class of failure.

5. New words before we use them

unit test = checks a small function; contract test = checks shape/rules at a boundary; integration test = checks components together; end-to-end test = checks a whole path; regression test = keeps a fixed bug from returning; fixture = controlled test input.

6. Useful video / source route

[Anuj Syal - Data Quality with Great Expectations - Testing/data-quality workflow demonstration](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-02 - Follow with me once

7. Follow with me once

From PARCELPULSE_PROJECT run `python -m pytest -q` and save the actual output in your attempt note. Then open `tests/test_m16.py`. For every test name, label the risk it covers: data quality, PII, secret leakage, environment policy, observability or incident behavior. Pick one function such as `mask_email` and explain why its unit test cannot prove GitHub permissions, network access or production freshness. Next invent one integration risk that the local unit tests do not exercise, such as a cloud identity failing to read a real storage path. Write the smallest test/evidence that would expose that risk. This separates “test count” from “risk coverage.”

```
fast/many: unit -> contract/data -> integration -> end-to-end :slow/few
production checks continue after deployment
```

8. What you should see / be able to say

The local suite passes 9 tests in the supplied baseline, and your risk matrix shows which failures are covered locally versus which need integration/live evidence.

9. Why it worked

Small tests give fast feedback and precise diagnosis; broader tests give confidence that boundaries cooperate. Production observations cover timing/data conditions that pre-deploy tests cannot reproduce fully.

10. Common beginner mistakes

Thinking 100% unit coverage proves production; using only happy-path test data; writing brittle end-to-end tests for every detail; not adding a regression test after an incident.

11. If you get stuck

If `pytest` is missing, run `python -m pip install pytest pyyaml` in your learning environment. If a test fails, read only the first failure, identify input/expected/actual, and do not edit the test merely to make green.

DE16-02 - Independent proof and mastery

12. You do a small version alone

CREATE `lesson_workspaces/DE16-02/attempt.md`. Classify all 9 existing tests by layer/risk and propose two missing tests: one fast local test and one integration/live check. Do not copy the labels from the guided example.

13. Validate

Every proposed test names the failure it catches, controlled input/setup, observable pass/fail signal, and whether it can run locally.

14. Teach it back

Explain why a green unit-test suite cannot prove cloud permissions or freshness, and when you would choose a slower integration test.

15. Protected review + changed retry

Save `attempt.md` first. Only then may you open the matching protected Review. If repair is assigned, preserve `attempt.md` append-only, close Review, and CREATE `retry.md` using this changed task: For ClinicOps, design a new 5-test strategy protecting patient-event privacy, schema contract, 20-minute freshness, deployment policy and rollback validation.

16. Evidence / mastery + exact next route

pytest output + `attempt.md` risk coverage. Do not fabricate live integration results. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-03.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-03 - Automated quality gates

1. Why this matters

Build automated quality gates that block unsafe data while keeping non-critical anomalies visible as warnings. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Build automated quality gates that block unsafe data while keeping non-critical anomalies visible as warnings.

3. Prerequisite / start state

You know validation, exit codes and data-quality controls from M09. ParcelPulse includes good and bad data plus a blocking gate.

4. Picture it in your head

Airport screening has hard stops and informational notices. A prohibited item blocks boarding; a slightly heavy bag may follow another process. If every notice blocks, operations become unusable. If hard stops only warn, unsafe output escapes. A data gate needs explicit severity, evidence, owner and recovery.

5. New words before we use them

quality gate = policy checkpoint; blocking check = must stop publish/change; warning = visible anomaly that may continue; threshold = defined boundary; exit code = process success/failure signal; quarantine = isolate invalid records.

6. Useful video / source route

[Anuj Syal - Data Quality with Great Expectations - Expectation/validation section; map to blocking vs warning gates](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-03 - Follow with me once

7. Follow with me once

Run the good quality gate and record its exit status plus metrics.json. Then run the bad quality gate; it is expected to return non-zero, so do not “fix” the command to make it green. Open rejected.json and connect each defect to a blocking rule. Now consider a different signal: today has 5% fewer rows than yesterday. Write two contexts where 5% should warn and one where it should block. Add owner and recovery action for each. Finally distinguish a record-level quarantine from a whole-batch stop. This shows why a quality gate is policy encoded in software, not a magic score such as 85%.

```
python src/quality_gate.py data/orders_good.csv evidence/good # expect 0
python src/quality_gate.py data/orders_bad.csv evidence/bad  # expect non-zero
```

8. What you should see / be able to say

Good data exits 0 with 8 accepted, 0 rejected and accepted amount 990. Bad data exits non-zero with 5 accepted, 3 rejected and accepted amount 685. Your severity policy explains warning vs block.

9. Why it worked

Explicit binary safety rules prevent invalid publication while evidence and ownership make the decision reviewable. Context-dependent warnings prevent harmless variation from becoming an outage.

10. Common beginner mistakes

Using an arbitrary overall score; treating every anomaly as fatal; allowing duplicate required keys; not preserving rejected rows; not defining who responds.

11. If you get stuck

If the expected non-zero command looks like failure, remember that this is the test. Read metrics.json and rejected.json. If counts differ, restore the original CSV and run from project root before editing anything.

DE16-03 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-03/attempt.md. Design five quality rules for a new delivery feed: at least three blockers and two warnings. Include evidence, owner and recovery for each. Run both supplied gate commands and record observed controls.

13. Validate

No percentage/average can override a failed blocker. Your policy makes severity explicit and the observed bad dataset deliberately fails.

14. Teach it back

Explain the difference between a quality rule and a quality gate, and give one anomaly whose severity depends on business context.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: create blocking/warning policy for duplicate visit IDs, missing contact, privacy leakage, 20-minute freshness and a 4% volume drop.

16. Evidence / mastery + exact next route

good/bad gate outputs + attempt.md; preserve the failing bad run as valid test evidence. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-04.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-04 - Continuous Integration fundamentals

1. Why this matters

Explain what Continuous Integration protects, when it runs, and what a green CI result cannot prove. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Explain what Continuous Integration protects, when it runs, and what a green CI result cannot prove.

3. Prerequisite / start state

You can commit code with Git and run local tests. You have not yet needed a hosted CI runner.

4. Picture it in your head

CI is a factory checkpoint applied consistently to every candidate change. The checkpoint can inspect what you teach it to inspect. It cannot see future production traffic, expired permissions tomorrow, or a downstream service that is currently outside the test environment.

5. New words before we use them

CI = automated validation of candidate changes; trigger = event starting a workflow; runner = machine executing jobs; job = isolated workflow unit; step = action/command; artifact = saved workflow output; status check = reported pass/fail.

6. Useful video / source route

[DP-700 Full Course - CI/CD & Lifecycle Management - 0:48:26 onward](#)

[GitHub Actions documentation - Current workflow behavior](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-04 - Follow with me once

7. Follow with me once

Open `.github/workflows/ci.yml` as plain text. Identify the push and pull_request triggers, the single test-and-policy job, the Ubuntu runner, and every step in order. Notice permissions: contents: read. Run locally the same three logical validations: `python -m pytest -q`, `python src/policy_checks.py .`, and `python src/secret_scan.py ..` Record that local parity is useful but does not equal a hosted GitHub run. Then invent a production problem that can occur after green CI, such as a revoked cloud role or stale source feed. State which post-deploy signal would catch it. You are separating pre-merge confidence from production health.

```
push / pull_request
-> checkout -> setup -> tests -> policy -> secret scan
-> PASS or BLOCK
```

8. What you should see / be able to say

You can trace trigger -> job -> steps -> result and explain at least one production failure that a green CI run does not rule out.

9. Why it worked

Automating deterministic checks prevents developers from skipping them and creates repeatable change evidence. Keeping production monitoring separate prevents false confidence.

10. Common beginner mistakes

Calling any workflow CI; allowing write permissions by default; assuming local YAML means GitHub executed it; putting production secrets directly in workflow text.

11. If you get stuck

If YAML is confusing, ignore indentation details first and mark only four labels: on, permissions, jobs, steps. Then use the exact file. If PyYAML interprets on oddly, remember GitHub Actions uses its own workflow schema; do not "fix" valid GitHub syntax just for a generic YAML parser.

DE16-04 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-04/attempt.md. Draw the current CI workflow as a five-box flow, list what each check blocks, and name two failures it cannot detect before deployment.

13. Validate

Your flow matches ci.yml exactly, permissions remain least-privilege, and live GitHub execution is labeled PENDING unless you observed a real run.

14. Teach it back

Explain CI in one minute without saying “it deploys automatically.” Include one thing CI proves and one thing it cannot prove.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design a CI candidate gate containing tests, privacy policy, secret scan and release manifest validation. Keep deployment separate.

16. Evidence / mastery + exact next route

local parity command outputs + workflow reading note. Hosted GitHub run remains separate runtime evidence. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-05.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-05 - GitHub Actions for data projects

1. Why this matters

Read and design GitHub Actions workflows with explicit triggers, permissions, environments and safer cloud authentication. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Read and design GitHub Actions workflows with explicit triggers, permissions, environments and safer cloud authentication.

3. Prerequisite / start state

You understand CI purpose from DE16-04 and Git/GitHub from M06.

4. Picture it in your head

A workflow is a written shift plan: when the shift starts, which workers run, which tools they use, and what keys each worker receives. GitHub permissions control the platform token; OIDC lets a trusted workflow request a short-lived identity proof for a cloud provider instead of storing a permanent cloud password.

5. New words before we use them

GITHUB_TOKEN = ephemeral repository-scoped token; OIDC = federated identity protocol; id-token: write = permission to request an OIDC token; subject/sub = identity claim describing workflow context; environment = protected deployment context; trust policy = cloud rule deciding which OIDC claims may assume a role.

6. Useful video / source route

[GitHub Actions documentation - Use current quickstart/workflow syntax](#)

[GitHub Actions secure use reference - Least privilege and workflow security](#)

[GITHUB_TOKEN concept - Repository-scoped ephemeral token](#)

[GitHub OIDC with cloud providers - OIDC identity/trust conditions and claim format](#)

[GitHub OpenID Connect reference - OIDC identity/trust conditions and claim format](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-05 - Follow with me once

7. Follow with me once

Compare `.github/workflows/ci.yml` with `deploy_oidc_example.yml.disabled`. The CI workflow needs only `contents: read`. The disabled deployment sketch also requests `id-token: write` because OIDC token minting needs that permission. Say aloud: “id-token: write does not grant arbitrary cloud resource write.” A cloud login action and cloud-side trust policy still decide what identity is assumed and what that identity may do. On current GitHub, trust conditions must match the actual OIDC token claims; new repositories created after July 15, 2026 may use immutable owner/repository IDs in the default subject. Do not memorize one subject string. Inspect the actual token/official docs and scope trust to the intended repository/environment. Keep the sample disabled because this course cannot invent your cloud account.

```
permissions:
  contents: read
  id-token: write # request OIDC token only; NOT arbitrary cloud write

jobs:
  deploy:
    environment: production
```

8. What you should see / be able to say

You can distinguish GitHub token permissions, OIDC token request permission, cloud trust policy, cloud RBAC, and environment protection as separate controls.

9. Why it worked

Short-lived federated credentials reduce long-lived secret exposure, while claim constraints and environment protection reduce who/what may deploy. Separation of controls limits blast radius.

10. Common beginner mistakes

Thinking `id-token: write` gives cloud write access; storing cloud client secrets in repository variables by default; using wildcard trust subjects; forgetting environment protection/reviewer rules.

11. If you get stuck

If OIDC feels abstract, draw four boxes: GitHub workflow -> OIDC token -> cloud trust policy -> cloud role permissions. Ask at each arrow “who verifies whom?” Leave the example disabled until you have a real authorized cloud target.

DE16-05 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-05/attempt.md. Annotate both workflow files: trigger, platform permissions, environment, OIDC request, missing cloud login/trust pieces. Propose a least-privilege trust scope without inventing a real credential.

13. Validate

Your note says id-token: write only permits token request; cloud access still depends on trust policy and role permissions. Live workflow/OIDC proof is PENDING unless actually observed.

14. Teach it back

Explain how OIDC replaces a stored cloud password, and why a broad trust policy can still be dangerous even with short-lived tokens.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design a protected production deployment identity flow using a GitHub environment, OIDC, constrained trust and least-privilege cloud role. Do not reuse ParcelPulse names.

16. Evidence / mastery + exact next route

workflow annotation + trust-design note. Real GitHub OIDC token/cloud login remains live-runtime pending. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-06.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-06 - Dev, test and prod environments

1. Why this matters

Separate dev, test and prod environments so experiments cannot silently alter production behavior or data. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Separate dev, test and prod environments so experiments cannot silently alter production behavior or data.

3. Prerequisite / start state

You can read YAML and environment-specific config files.

4. Picture it in your head

Think of three kitchens using the same recipe but different rooms, ingredients and permissions. Code may be promoted, but production data, credentials and target names should not be casually shared. Environment separation lets you test changes without pretending test conditions equal production.

5. New words before we use them

environment = isolated execution context; config = non-secret behavior settings; promotion = move a tested change toward production; parity = important similarity across environments; drift = unintended differences; segregation = separation of access/resources.

6. Useful video / source route

[Microsoft Reactor - CI/CD in Microsoft Fabric: From Development to Frictionless Production - Phase 2 - 3:26-11:35](#) [Git/Azure DevOps integration, deployment pipelines, Dev/Test/Prod architecture](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-06 - Follow with me once

7. Follow with me once

Open config/dev.yml, test.yml and prod.yml side by side. Identify what is intentionally different and what should remain structurally consistent. Run `python src/policy_checks.py`; one protected rule proves the three targets are distinct. Now imagine someone changes test.yml to point at the prod target. Predict the risk before running anything. Next design a setting that may differ safely, such as sample size or alert channel, and a property that should not differ, such as required schema validation. Finally explain why secrets should be referenced through environment-specific secret stores rather than copied between config files. Save observed local policy output, not an invented cloud screenshot.

```
config/dev.yml -> target: parcelpulse-dev
config/test.yml -> target: parcelpulse-test
config/prod.yml -> target: parcelpulse-prod
```

8. What you should see / be able to say

Distinct dev/test/prod targets and a written separation policy naming data, identity, config and deployment controls.

9. Why it worked

Isolation limits blast radius and makes promotion meaningful. Deliberate parity reduces “works in test, fails in prod” surprises without forcing production secrets/data into test.

10. Common beginner mistakes

Using the same database for all environments; copying prod credentials to dev; hiding important config differences; promoting code without recording the candidate version.

11. If you get stuck

If config differences are unclear, compare only the top-level keys first. Ask: should this key be identical, intentionally different, or secret/not stored here? Run `policy_checks.py` from project root to confirm current baseline.

DE16-06 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-06/attempt.md. Build a three-column dev/test/prod matrix for target, data, identity, secrets, alerting and approval. State why each is same or different.

13. Validate

Targets are distinct, secrets are referenced rather than literal, and your matrix preserves the same correctness controls across environments.

14. Teach it back

Explain environment separation as risk control, not as “three folders.” Give one example of dangerous drift and one example of intentional difference.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design dev/test/prod separation where test cannot access real patient identifiers and production deployment needs protected approval.

16. Evidence / mastery + exact next route

attempt.md + local policy check output; cloud environment configuration only if genuinely observed. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-07.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-07 - Deployment, releases and rollback

1. Why this matters

Design release, deployment, post-deploy validation and rollback as one controlled change process. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Design release, deployment, post-deploy validation and rollback as one controlled change process.

3. Prerequisite / start state

You understand CI and environment promotion. ParcelPulse has a release manifest and rollback plan.

4. Picture it in your head

A deployment is crossing a bridge with a return plan. You do not burn the bridge behind you before checking the destination. A release has an identity, preconditions, observable post-deploy health, and a safe rollback target whose data compatibility must be considered.

5. New words before we use them

release = versioned candidate delivered to an environment; deployment = act of applying release; rollback = restore prior application/config version; roll-forward = repair with a new version; migration = change to durable data/schema; post-deploy check = health validation after change.

6. Useful video / source route

[Microsoft Reactor - CI/CD in Microsoft Fabric: From Development to Frictionless Production - Phase 2 - 4:16-11:35 and 52:35-57:48 deployment flow and automation](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-07 - Follow with me once

7. Follow with me once

Open `release/release_manifest.json` and `release/rollback_plan.md`. Identify current, candidate and rollback versions plus the four post-deploy checks. Walk through a simulated failure: candidate v1.5.0 deploys, `quality_gate` fails, and you suspect the release. Step 1 freeze further deployments; step 2 preserve the failed evidence; step 3 restore the known-good artifact/config; step 4 inspect whether any data migration is reversible; step 5 rerun quality, reconciliation, freshness and PII checks. Write why “git revert” and “production rollback” are related but not identical. A code revert does not automatically undo data state.

```
candidate v1.5.0 -> deploy -> post checks
                        | FAIL
                        v
freeze + preserve evidence -> rollback v1.4.0 -> revalidate
```

8. What you should see / be able to say

A release decision with version, trigger, rollback target, preserved evidence and explicit post-rollback validation. You mention data compatibility before reversing anything.

9. Why it worked

Versioned releases make changes traceable; health checks detect harmful deployment effects; rollback reduces recovery time only when it is rehearsed and safe for durable data.

10. Common beginner mistakes

Deleting failed evidence; blindly reversing data migrations; calling deploy success before health checks; having no rollback version; rolling back without revalidation.

11. If you get stuck

If release terms blur, point to three different artifacts: Git commit, release manifest, running environment. They may align, but they are not automatically the same object. Follow `rollback_plan.md` exactly before inventing new steps.

DE16-07 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-07/attempt.md. Write a release/rollback decision tree for a failed post-deploy quality check, including when you would roll forward instead.

13. Validate

The decision tree preserves evidence, names a known-good target, protects durable data, and ends with post-recovery validation.

14. Teach it back

Explain why rollback is not “undo everything,” and why a successful deployment command is not enough to call the release healthy.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: candidate v2.3 fails a privacy check after deployment. Design containment, rollback/roll-forward choice and validation without reusing ParcelPulse versions.

16. Evidence / mastery + exact next route

release decision + actual local manifest/plan inspection. Live deployment remains pending. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-08.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-08 - Secrets and credential safety

1. Why this matters

Prevent credential leakage and design safer secret handling with scanning, rotation/revocation and short-lived identity where possible. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Prevent credential leakage and design safer secret handling with scanning, rotation/revocation and short-lived identity where possible.

3. Prerequisite / start state

You can read workflow/config files and understand OIDC from DE16-05.

4. Picture it in your head

A secret is a physical key. Hiding the key under a doormat is not security; copying it into code creates more doormats. If a key leaks, removing the photo of it is not enough—you must invalidate the key. Short-lived identity reduces how long a stolen credential remains useful.

5. New words before we use them

secret = credential/private value; secret scanning = detect known secret patterns; push protection = block likely secrets before upload; rotate = replace credential; revoke = invalidate credential; vault/secret store = managed secret service; credential lifetime = how long value remains usable.

6. Useful video / source route

[GitHub - What is GitHub Secret Protection? | GitHub Explained - Official explainer](#)

[GitHub push protection - Current leak-prevention behavior](#)

[GitHub secret scanning - Current scanning behavior](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-08 - Follow with me once

7. Follow with me once

Run `python src/secret_scan.py .` and record a clean result. In a temporary copy only, create a text file containing a fake DEMO-TOKEN- followed by a sufficiently long demo value; rerun the scanner and confirm it detects the file. Delete the temporary copy. Now walk through a real leak response without using a real secret: revoke/rotate the exposed credential first, remove it from code/history as appropriate, inspect access logs, then add prevention. Compare repository secrets with OIDC: a stored secret still exists and can leak, while OIDC exchanges trusted workflow identity for a short-lived token. Neither removes the need for least privilege.

```
python src/secret_scan.py .  
# clean project -> exit 0  
# injected DEMO-TOKEN-... in temp copy -> non-zero
```

8. What you should see / be able to say

Clean baseline scan, deliberate fake-token detection, and a response sequence that starts with revoke/rotate rather than merely deleting the line.

9. Why it worked

Scanning catches some leaks, but credential invalidation limits actual exposure. Short-lived identity shrinks persistence, and least privilege limits impact if authentication is abused.

10. Common beginner mistakes

Committing `.env`; printing secrets to logs; deleting leaked text without revoking; pasting real credentials into a “scanner test”; assuming secret scanning catches every secret format.

11. If you get stuck

Never create a real token for this lesson. If the demo scanner reports a false hit, open only the reported file/line. If it misses the fake token, verify the demo string matches the documented DEMO-TOKEN pattern before changing the scanner.

DE16-08 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-08/attempt.md. Write a six-step incident response for an accidentally committed API key and design a safer authentication replacement. Use fake names only.

13. Validate

Your first containment includes revoke/rotate, no real value is saved, and the prevention plan covers storage, scanning, permissions and short-lived identity where supported.

14. Teach it back

Explain why removing a leaked secret from the latest commit is not sufficient, and how OIDC changes credential lifetime.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: respond to a leaked demo storage credential in a protected deployment workflow, including patient-data access review and a safer identity design.

16. Evidence / mastery + exact next route

clean scanner output + fake-detection proof + response design; never include real secret values. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-09.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-09 - RBAC and least privilege

1. Why this matters

Design RBAC using least privilege, explicit scope and separation of duties. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Design RBAC using least privilege, explicit scope and separation of duties.

3. Prerequisite / start state

You understand identity vs authorization from M15 and environment separation from DE16-06.

4. Picture it in your head

An office badge should open only the rooms required for a job. A reader does not need the deployment room; a deployer may not need raw customer records. RBAC expresses those boundaries as role permissions applied to identities at a defined scope.

5. New words before we use them

RBAC = role-based access control; role = named permission bundle; principal = user/service identity; action = permitted operation; resource = protected object; scope = boundary where permission applies; least privilege = minimum access needed; separation of duties = split risky powers.

6. Useful video / source route

[Microsoft Learn - Manage and right-size RBAC role assignments - Least-privilege roles and scope](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-09 - Follow with me once

7. Follow with me once

Open security/roles.json. For each role, say who might need it, what actions are allowed and where the scope ends. Run python src/policy_checks.py . and confirm there is no wildcard action. Now test the design mentally: should data-reader deploy? Should deployer automatically read raw customer email? If not, explain why. Create one deliberately overpowered role on paper with action "*" and explain how the policy check should reject it. Then design a break-glass process conceptually: temporary elevated access, approval, audit and automatic expiry—not a permanent admin role.

```
data-reader: dataset:read @ curated/*
pipeline-writer: dataset:read/write + run:write @ parcelpulse/*
deployer: release:read/deploy/rollback @ parcelpulse-prod
```

8. What you should see / be able to say

A least-privilege role matrix with no wildcard permissions and a clear separation between data use, pipeline execution and deployment.

9. Why it worked

RBAC reduces blast radius by limiting what a compromised or mistaken identity can do. Scope and separation of duties matter as much as the role name.

10. Common beginner mistakes

Giving Contributor/Admin "just to make it work"; wildcard actions; mixing dev and prod scopes; granting human permissions to service identities indefinitely.

11. If you get stuck

If actions and scope blur, rewrite each grant as a sentence: "principal may ACTION on RESOURCE within SCOPE." If the sentence is broader than the job, reduce it. Use roles.json as the concrete example.

DE16-09 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-09/attempt.md. Design roles for analyst-reader, pipeline-service and production-deployer for a new parcel region. Include one explicit denied action per role.

13. Validate

No role has wildcard actions, prod scope is narrow, and risky powers are separated. Local policy_checks.py remains green.

14. Teach it back

Explain least privilege using action + resource + scope, and why a role named “reader” can still be dangerous if its scope is too broad.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design minimum roles for reporting analyst, ingestion service and release operator where patient identifiers require narrower access.

16. Evidence / mastery + exact next route

role matrix + local policy output. Real cloud role assignments only if genuinely configured. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-10.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-10 - PII, encryption and masking

1. Why this matters

Protect PII through minimization, masking and appropriate encryption boundaries while preserving useful data. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Protect PII through minimization, masking and appropriate encryption boundaries while preserving useful data.

3. Prerequisite / start state

You understand schemas, security roles and Python transformations.

4. Picture it in your head

Sensitive data is like a document containing private details. Minimization asks whether you need the page at all. Masking changes what a viewer sees. Encryption protects bytes from someone who lacks the key. These are different controls and can be combined.

5. New words before we use them

PII = personally identifiable information; minimization = keep only necessary data; masking = hide part/all of displayed value; tokenization = replace sensitive value with surrogate; encryption at rest = protect stored bytes; encryption in transit = protect network traffic; key management = control encryption keys.

6. Useful video / source route

[Microsoft Developer - Protect Sensitive Data with Azure AI Language PII Redaction - 00:58 overview; 1:12 demo; 1:42 masking styles; 2:16 exclusions](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-10 - Follow with me once

7. Follow with me once

Run `python src/pipeline.py data/orders_good.csv --out evidence/DE16-10_pii`. Open `curated_orders.csv` and verify raw email values are not present while `masked_email` is. Search specifically for `alice@example.com` and then for `a***@example.com`. Record observed results. Now classify three controls: masking protects what downstream users see; TLS/encryption protects transport/storage; RBAC restricts who can access the underlying asset. Decide whether downstream analytics needs raw email at all. If not, minimization is stronger than merely encrypting a value everyone later decrypts. Finally, write which team or role would be allowed to access any retained raw identifier and why that access is necessary.

```
alice@example.com -> a***@example.com
raw email should NOT appear in curated_orders.csv
```

8. What you should see / be able to say

The curated output contains masked email such as `a***@example.com` and does not contain raw `alice@example.com`. Your design distinguishes masking, access control and encryption.

9. Why it worked

Different controls protect different threat paths. Minimizing raw sensitive data reduces exposure surface; masking reduces unnecessary visibility; encryption helps protect bytes when storage or transport is accessed without keys.

10. Common beginner mistakes

Calling hashing encryption; storing encryption keys beside ciphertext; keeping raw PII “just in case”; assuming masking removes the need for access control; logging PII during errors.

11. If you get stuck

If the raw email still appears, confirm you opened `curated_orders.csv` rather than the input file. Trace `mask_email` in `src/pipeline.py`. Do not edit the source CSV to make the search pass.

DE16-10 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-10/attempt.md. Build a four-column PII treatment table: field, business need, allowed viewers, protection/minimization. Include email, phone, order ID and region.

13. Validate

Every retained sensitive field has a stated purpose and access boundary. Your local curated output proves at least one real masking control.

14. Teach it back

Explain why masking and encryption are not synonyms, and why deleting an unnecessary sensitive field can be safer than encrypting it.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design treatment for patient email, appointment ID, clinic region and free-text notes; minimize before choosing masking/encryption.

16. Evidence / mastery + exact next route

curated masking/search proof + PII treatment table. Never save real personal data in the course evidence. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-11.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-11 - Metadata, lineage and governance

1. Why this matters

Use metadata and lineage to understand ownership, meaning, classification and downstream impact before changing data. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Use metadata and lineage to understand ownership, meaning, classification and downstream impact before changing data.

3. Prerequisite / start state

You know datasets/tables and pipeline stages. ParcelPulse has a small catalog and lineage graph.

4. Picture it in your head

A city map tells you where roads go; a property register tells you who owns each plot and what it is. Lineage is the road map of data movement. Metadata/catalog information adds owner, meaning, grain, classification and freshness expectations so an engineer can judge impact.

5. New words before we use them

metadata = data about data; catalog = searchable inventory; lineage = upstream/transformation/downstream relationships; classification = sensitivity/business category; owner = accountable team/person; grain = what one row represents; impact analysis = identify consumers affected by change.

6. Useful video / source route

[Microsoft Learn Live - Microsoft Purview Data Governance - 1:46 overview; 24:54 scanning; 40:03 lineage; 52:26 classification](#)
[Microsoft Purview Data Map - Current metadata/data-map model](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-11 - Follow with me once

7. Follow with me once

Open governance/catalog.json and lineage.json. Read the catalog first: dataset, owner, grain, classification and freshness SLA. Then trace every lineage edge from source to transformation to curated output to analytics consumer. Imagine changing amount from numeric to text. Before changing code, list which nodes and consumers require impact analysis. Now imagine the owner field is missing: explain why a technically accurate lineage graph still leaves an operational gap. Finally add on paper one metadata field you would want for a real production dataset, such as retention, business definition or steward.

```
orders_good.csv -> pipeline.py -> curated_orders.csv -> analytics/orders_kpi
Catalog: owner=data-platform, grain=one accepted order,
classification=internal/masked
```

8. What you should see / be able to say

A correct source-to-consumer lineage trace plus owner/grain/classification/freshness metadata and a concrete impact analysis for one schema change.

9. Why it worked

Lineage makes dependency impact visible; catalog metadata makes the dependency understandable and actionable. Together they reduce blind changes and shorten incident investigation.

10. Common beginner mistakes

Treating lineage as a pretty diagram; cataloging without owners; confusing table schema with business meaning; allowing stale metadata to look authoritative.

11. If you get stuck

Start from one node only: input CSV. Follow each edge in lineage.json in order. Then return to catalog.json and ask who owns the final dataset and what one row means. Do not try to learn a full governance platform first.

DE16-11 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-11/attempt.md. Draw the ParcelPulse lineage from memory, then add an owner, grain, classification, freshness target and one impact-analysis question for every node.

13. Validate

Your graph has directional upstream/downstream edges and the metadata is sufficient for another engineer to find the owner and understand the row meaning.

14. Teach it back

Explain the difference between schema, metadata and lineage, and how lineage helps before a breaking change.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design catalog metadata and lineage for visit_events -> validation/masking -> curated_visits -> clinic_wait_time dashboard.

16. Evidence / mastery + exact next route

attempt.md lineage + catalog interpretation. Live Purview scanning remains optional/live-platform evidence only. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-12.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-12 - Observability for data pipelines

1. Why this matters

Observe pipeline health using status, freshness, volume, quality and latency signals tied to actionable thresholds. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Observe pipeline health using status, freshness, volume, quality and latency signals tied to actionable thresholds.

3. Prerequisite / start state

You understand quality gates and freshness from earlier modules.

4. Picture it in your head

A car dashboard uses different gauges because “engine running” does not prove fuel, temperature and speed are healthy. A data pipeline also needs multiple signals. A green job can still publish zero rows, stale data or poor-quality records.

5. New words before we use them

observability = infer system health from signals; freshness = age of newest expected data; volume = row/event count; quality = validity/reconciliation; latency = processing delay; SLI = measured indicator; SLO = target for indicator; alert = actionable threshold notification.

6. Useful video / source route

[DP-700 Full Course - Monitoring & Optimization - 4:46:53-5:43:50](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-12 - Follow with me once

7. Follow with me once

Run the observability command with reference time 09:45 and 30-minute SLA. Read every field: row_count, freshest_event_time, freshness_age_minutes, rejected_rows, latency_ms and healthy. Then rerun with reference time 12:45. The same file should now be stale because time changed, proving health depends on context. Write which signal would detect each failure: source stopped sending, duplicates increased, transformation slowed, job failed outright, or output arrived but contains too few rows. For each alert, add an owner/action; an alert that nobody can act on is noise.

```
python src/observability.py data/orders_good.csv 2026-09-10T09:45:00 30 # healthy
python src/observability.py data/orders_good.csv 2026-09-10T12:45:00 30 #
stale/non-zero
```

8. What you should see / be able to say

Healthy at the near reference time and unhealthy/stale at the later reference time, plus a multi-signal alert matrix with owners/actions.

9. Why it worked

Different failure modes change different signals. Combining job status with data freshness/volume/quality and latency prevents "green but wrong" pipelines from hiding.

10. Common beginner mistakes

Alerting on every metric; no owner; freshness measured from processing time instead of source/event time; using job success as the only health signal; thresholds with no business context.

11. If you get stuck

If the stale command exits non-zero, that is expected evidence. Read the JSON rather than treating the shell status as a broken script. If times confuse you, subtract freshest_event_time from reference_time manually.

DE16-12 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-12/attempt.md. Design five alerts for ParcelPulse with signal, threshold, severity, owner and first response. Include at least one data-health alert not based on job status.

13. Validate

Your alert set covers status, freshness, volume/quality and latency, and each page has an actionable response rather than a vague "investigate."

14. Teach it back

Explain why a successful scheduled job can still deliver bad or stale data, and which signal would expose that.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design observability for a 20-minute freshness SLO and patient-event quality, including one warning and one paging condition.

16. Evidence / mastery + exact next route

healthy/stale observed JSON + alert matrix. Preserve expected non-zero stale run as valid evidence. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-13.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-13 - Incidents, runbooks and root cause

1. Why this matters

Handle incidents with triage, containment, recovery, validation and evidence-based root-cause analysis. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Handle incidents with triage, containment, recovery, validation and evidence-based root-cause analysis.

3. Prerequisite / start state

You can read observability signals, release manifests and rollback plans.

4. Picture it in your head

A fire alarm is not the root cause of a fire. During an incident you first protect people and stop spread, then restore safe service, then investigate cause from evidence. Guessing root cause too early can send recovery in the wrong direction.

5. New words before we use them

incident = unplanned degradation/failure; severity = impact/urgency level; triage = establish scope/priority; containment = limit damage; mitigation = reduce impact; recovery = restore service; RCA = root-cause analysis; runbook = repeatable response steps; postmortem = learning record.

6. Useful video / source route

[Microsoft Azure Friday - Azure SRE Agent: Less Toil, More Uptime. Maximum Innovation - Incident workflow, runbook automation, diagnosis/root cause, post-deployment health](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-13 - Follow with me once

7. Follow with me once

Run `python src/incident.py post_deploy_quality_failed` and open the created incident record. Identify severity, symptom, owner, containment, rollback target, validation checks, `root_cause_status` and follow-up. Notice the root cause is deliberately unknown. Walk the timeline: detect quality failure, freeze further deployment, preserve evidence, restore/recover, rerun quality/reconciliation/freshness/PII checks, then investigate. Write two hypotheses but label them hypotheses, not facts. State what evidence would confirm or reject each. This practice separates urgent recovery from careful diagnosis and makes the later RCA defensible.

```
detect -> triage -> contain -> recover -> validate -> RCA -> prevention
root_cause_status = "unknown - investigate; do not guess"
```

8. What you should see / be able to say

A SEV2-style incident record with named owner, containment, rollback target, validation list and root cause explicitly unknown until evidence confirms it.

9. Why it worked

Containment/recovery reduces harm quickly; validation proves service is actually safe again; delaying RCA claims until evidence prevents false stories and weak preventive actions.

10. Common beginner mistakes

Guessing root cause during panic; deleting logs/evidence; fixing before defining impact; closing after one green rerun; writing “human error” without system-level prevention.

11. If you get stuck

If incident terms blur, use the seven-arrow sequence above. Ask “Are we reducing impact now, proving recovery now, or explaining why it happened?” Put each action in one category.

DE16-13 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-13/attempt.md. Write a runbook for a post-deploy data-quality failure with detection, owner, containment, recovery, validation, two hypotheses and evidence needed for RCA.

13. Validate

The runbook does not claim an unproven cause, preserves failed evidence, and requires post-recovery data health checks before closure.

14. Teach it back

Explain why recovery can happen before root cause is known, and why “we rolled back” does not prove data is healthy.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: respond to a privacy leak alert after a release. Include containment of access, credential review, safe recovery, validation and evidence-led RCA.

16. Evidence / mastery + exact next route

generated incident record + independent runbook; do not overwrite the original incident evidence. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: DE16-14.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

DE16-14 - Terraform and IaC foundations

1. Why this matters

Explain Infrastructure as Code and safely read Terraform configuration, plan/apply workflow and state risk. In production, this protects correctness, security, availability or recovery when the happy path stops being happy.

2. What you are learning

Explain Infrastructure as Code and safely read Terraform configuration, plan/apply workflow and state risk.

3. Prerequisite / start state

You understand environments, permissions and release safety. Terraform CLI may or may not be installed on your machine.

4. Picture it in your head

Instead of verbally telling someone how to assemble infrastructure every time, IaC writes the desired structure as versioned instructions. Terraform compares desired configuration with known/current state and proposes a plan. The plan is a change preview, not permission to apply blindly. State is operationally sensitive because it maps configuration to real resources.

5. New words before we use them

IaC = infrastructure as code; provider = plugin/API integration; resource = managed object; variable = configurable input; output = exposed result; state = Terraform mapping/metadata about managed resources; init = initialize working directory; validate = syntax/config check; plan = proposed changes; apply = execute approved plan; drift = real state differs from desired.

6. Useful video / source route

[HashiCorp - Terraform Basics - 0:00-15:30](#)

[HashiCorp - Install Terraform - Current stable CLI - 1.16.2 at build audit](#)

[Terraform language documentation - Current HCL/configuration authority](#)

While watching/reading: Use only the named segment/concept. Sketch control -> evidence -> failure -> recovery, then close the source before independent work.

DE16-14 - Follow with me once

7. Follow with me once

Open terraform/main.tf, variables.tf, outputs.tf and .gitignore without applying anything. Identify provider/resource/variable/output/state-related files. If Terraform 1.16.x is actually installed, run terraform fmt -check, terraform init, terraform validate and terraform plan only against the safe supplied classroom configuration; save exact output and version. If it is not installed, record LIVE_TERRAFORM_PENDING rather than fabricating success. Regardless, read the proposed workflow: configuration -> fmt/validate -> init -> plan -> human/policy review -> apply -> post-change validation. Explain why state must be protected and why a plan can still be dangerous if credentials permit changing the wrong account.

```
terraform fmt -check
terraform init
terraform validate
terraform plan
# inspect plan + permissions before apply
```

8. What you should see / be able to say

A correct Terraform workflow explanation and, only when CLI really exists, genuine fmt/init/validate/plan evidence with terraform version. No fabricated apply.

9. Why it worked

Versioned desired-state configuration makes infrastructure changes reviewable and repeatable, while plans expose intended changes before execution. Protected state and scoped credentials reduce accidental cross-environment damage.

10. Common beginner mistakes

Treating plan as harmless; committing state/secrets; running apply just to get evidence; using production credentials in a learning project; assuming IaC eliminates the need for monitoring/rollback.

11. If you get stuck

If Terraform is absent, stop after inspecting files and record the pending runtime. Do not install random binaries from unofficial links. Use the official install page later. If init fails, read provider/network error before editing HCL.

DE16-14 - Independent proof and mastery

12. You do a small version alone

CREATE lesson_workspaces/DE16-14/attempt.md. Explain every supplied Terraform file, write the safe command order, identify three state/credential risks, and record whether real CLI execution is OBSERVED or PENDING.

13. Validate

No real apply is required for course artifact QA. A runtime claim includes actual terraform version and command output; otherwise it stays PENDING. Your design keeps state and credentials out of source control.

14. Teach it back

Explain desired state, plan and state to a beginner, and why "Terraform file exists" is not proof that infrastructure was successfully created.

15. Protected review + changed retry

Save attempt.md first. Only then may you open the matching protected Review. If repair is assigned, preserve attempt.md append-only, close Review, and CREATE retry.md using this changed task: ClinicOps: design an IaC change review for a test-to-prod storage resource, including environment scope, plan review, state protection and rollback/forward validation.

16. Evidence / mastery + exact next route

attempt.md + actual Terraform CLI outputs if available. PENDING is correct when the CLI/cloud target is absent. Master this lesson only when the evidence is inspectable and the teach-back is correct. Next route: M16 Module Practical, then M17.

No manual pass

A checkbox or status change cannot repair missing proof. Evidence + validation + changed proof after repair are required.

M16 Mastery Checklist

C1 Production readiness	control chain + ownership/recovery
C2 Testing strategy	risk-layered tests + observed pytest
C3 Quality gates	good passes; bad deliberately blocks; severity policy
C4 CI / GitHub Actions	workflow anatomy + least privilege; live run separate
C5 Environments + releases	dev/test/prod separation + rollback/revalidation
C6 Secrets/OIDC	clean scan + fake detection + safe identity model
C7 RBAC + PII	least privilege + masking/minimization evidence
C8 Metadata/lineage	catalog + source-to-consumer impact trace
C9 Observability/incidents	healthy/stale proof + runbook/RCA discipline
C10 IaC/Terraform	desired-state reasoning; live CLI proof only if observed

Move-on rule

ALL C1-C10 must be DEMONSTRATED at the required level with inspectable evidence and ZERO unresolved critical failures. No percentage or average shortcut. Live-only claims remain PENDING until genuinely observed.

M16 Revision / Quick Recall

Production ready = repeatable + tested + validated + secured + observable + recoverable + owned.

CI is pre-change automation; observability is post-change operational truth. A green CI run cannot prove production freshness or permissions.

Quality gate = explicit blocker/warning policy + evidence + owner + recovery. Do not hide a failed critical rule inside an average score.

Secrets: if leaked, revoke/rotate first. OIDC reduces long-lived secret storage but still needs constrained trust + least-privilege cloud role.

Incident: detect -> triage -> contain -> recover -> validate -> RCA -> prevention. Unknown root cause is better than a confident guess.

Terraform: config -> fmt/validate -> init -> plan -> review -> apply -> post-change validation. State and credentials are sensitive.

Next route

After the competency-based M16 practical, proceed to M17. G06 remains locked until M15 + M16 + M17 are all complete.